

使用 ADK 評估代理

來源：<https://codelabs.developers.google.com/adk-eval/instructions?hl=zh-tw>

步驟數：11

瞭解如何生成黃金資料集及執行評估，確保 AI 代理值得信賴。

目錄

1. 信任缺口
2. 設定
3. 產生黃金資料集 (adk web)
4. 匯出最佳資料集
5. 評估檔案
6. 針對黃金資料集執行評估 (adk eval)
7. 建立自訂測試
8. 執行自訂測試的評估作業 (adk eval)
9. (選用：僅供讀取) - 疑難排解與偵錯
10. 使用 Pytest (pytest) 持續整合/持續推送軟體更新
11. 結論

步驟 1 · 約 0 分鐘

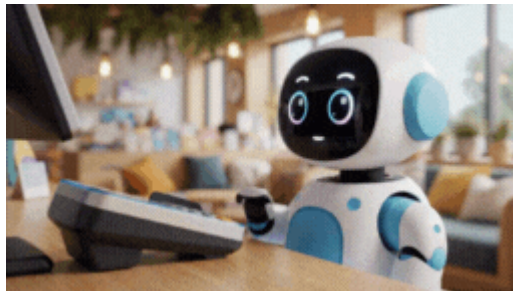
1. 信任缺口

靈感湧現的時刻

您建立了一個**客服專員**，這項功能適用於你的電腦。但昨天，它卻告訴顧客缺貨的智慧手錶有現貨，甚至还編造退款政策。知道服務專員正在工作，你晚上睡得著嗎？

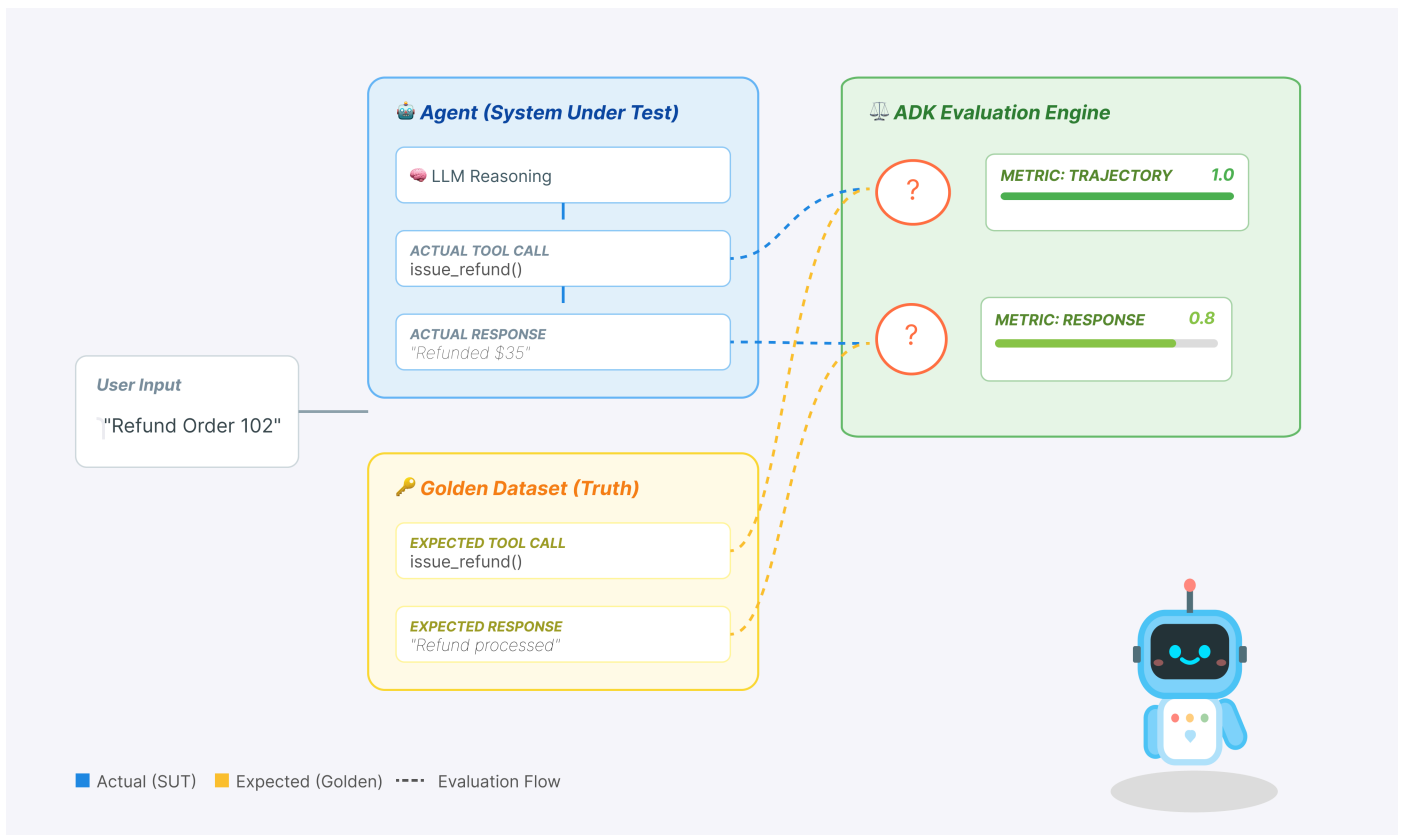
問題：生成式 AI 具有不確定性。與傳統軟體不同，您無法編寫簡單的 `if` 陳述式來檢查正確性。無法評估就無法修正。否則就只能盲目行事。

要將 AI 代理從概念驗證做到可正式部署，關鍵在於建立完善的自動化評估框架。



我們實際評估的是什麼？

代理程式評估比標準 LLM 評估更複雜。您不只是評估文章 (最終回覆)，而是評估**數學** (用來得出最終回覆的邏輯/工具)。



1. **軌跡 (程序)：**服務專員是否在適當的時機使用合適的工具？是否在 `place_order` 之前呼叫 `check_inventory` ？
2. **最終回覆 (輸出內容)：**答案是否正確、禮貌，且根據資料生成？

重要概念：軌跡與回應

- 軌跡評估：確保代理程式遵循商業邏輯 (例如「一律先驗證使用者身分，再檢查餘額」)。這基本上就是邏輯/迴歸測試。
- 回覆評估：確保語言品質和準確度。這就是品質查驗。

開發生命週期

在本程式碼研究室中，我們將逐步瞭解專業的代理程式測試生命週期：

- 本機目視檢查 (ADK 網頁式 UI)：手動對話並驗證邏輯 (步驟 1)。
- 單元/迴歸測試 (ADK CLI)：在本機執行特定測試案例，以快速找出錯誤 (步驟 3 和 4)。
- 偵錯 (疑難排解)：分析失敗原因並修正提示邏輯 (步驟 5)。
- 整合 CI/CD (Pytest)：在建構管道中自動執行測試 (步驟 6)。

2. 設定

如要為 AI 代理提供支援，我們需要兩項要素：Google Cloud 專案做為基礎。

第一部分：啟用帳單帳戶

如要執行本程式碼研究室，您需要有具備部分抵免額的帳單帳戶。請使用本程式碼研究室頂端橫幅中的抵免額開始操作。如果已連結帳單帳戶，可以略過這個步驟。

第二部分：開放式環境

1. 🖱️ 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
2. 🖱️ 如果系統在今天任何時間提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

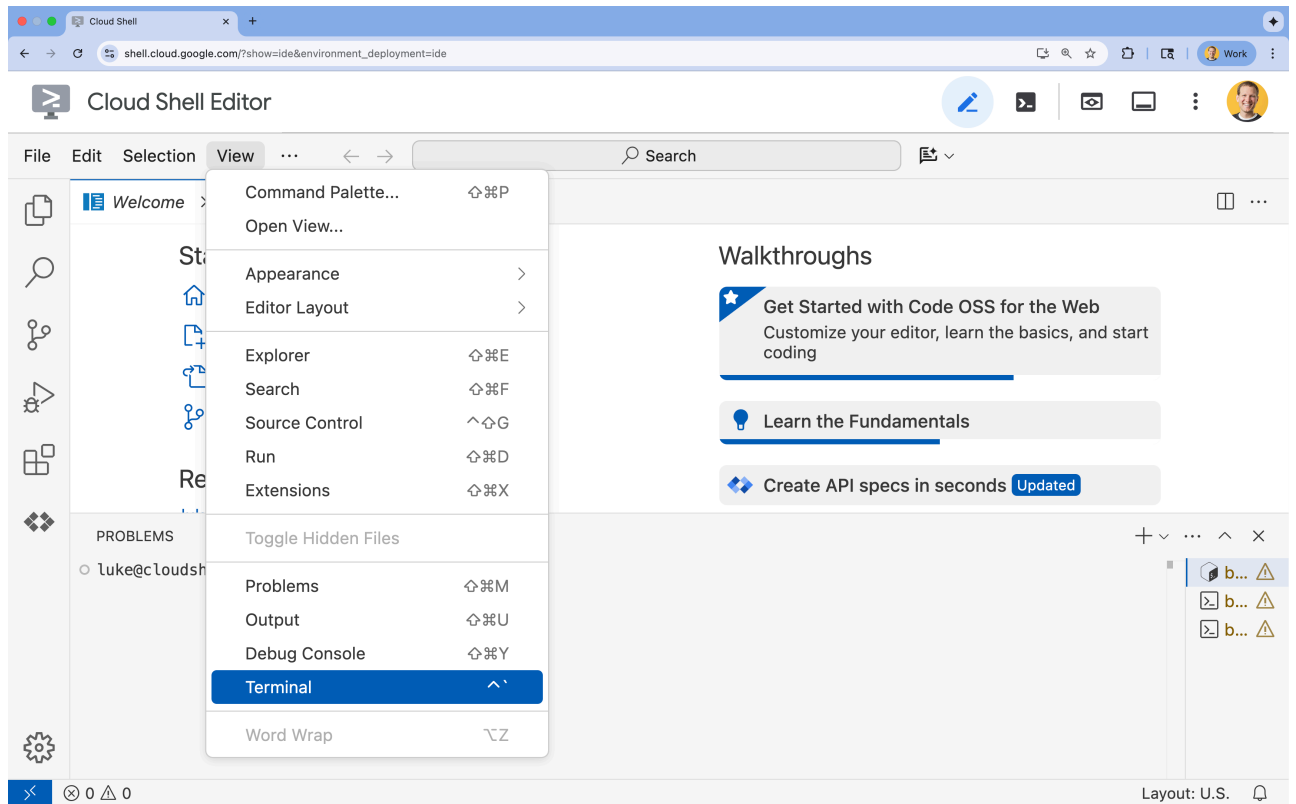
gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

3. 🖱️ 如果畫面底部未顯示終端機，請開啟終端機：
 - 按一下「查看」
 - 按一下「終端機」



4. 🖱️ 在終端機中，使用下列指令驗證您是否已通過驗證，以及專案是否已設為您的專案 ID：

```
gcloud auth list
```

5. 🖱️ 從 GitHub 複製啟動程序專案：

```
git clone https://github.com/cuppibla/adk_eval_starter
```

6. 🖱️ 從專案目錄執行設定指令碼。

⚠️ **專案 ID 注意事項：**指令碼會建議隨機產生的預設專案 ID。您可以按下 **Enter** 鍵接受這項預設值。不過，如果您想建立特定新專案，可以在指令碼提示時輸入所需的專案 ID。

```
cd ~/adk_eval_starter
./init.sh
```

指令碼會自動處理其餘設定程序。

7. 🖱️ 設定所需的專案 ID：

```
gcloud config set project $(cat ~/project_id.txt) --quiet
```

第三部分：設定權限

1. 🖱️ 使用下列指令啟用必要的 API。這項作業可能需要幾分鐘才能完成。

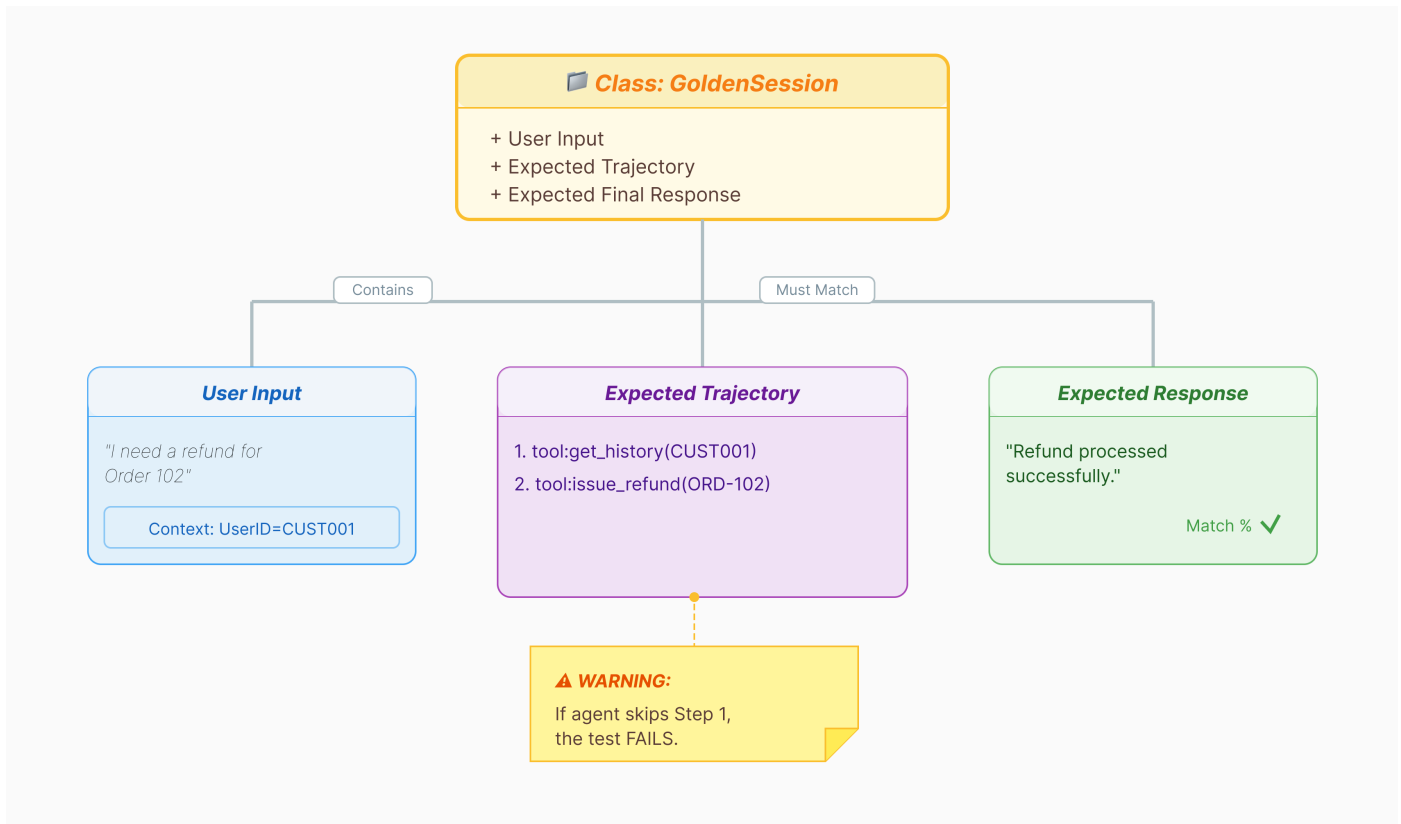
```
gcloud services enable \  
  cloudresourcemanager.googleapis.com \  
  servicenetworking.googleapis.com \  
  run.googleapis.com \  
  cloudbuild.googleapis.com \  
  artifactregistry.googleapis.com \  
  aiplatform.googleapis.com \  
  compute.googleapis.com
```

2. 🖱️ 在終端機中執行下列指令，授予必要權限：

```
. ~/adk_eval_starter/set_env.sh
```

請注意，系統會為您建立 `.env` 檔案。顯示專案資訊。

3. 產生黃金資料集 (adk web)



我們需要答案鍵才能評估服務專員。在 ADK 中，這稱為「黃金資料集」。這個資料集包含「完美」的互動，可做為評估的基準真相。

什麼是黃金資料集？

黃金資料集是代理程式正確執行的快照。這不只是問答配對清單，這項功能會擷取：

- 使用者查詢 (「我要退款」)
- 軌跡 (工具呼叫的確切順序：`check_order` -> `verify_eligibility` -> `refund_transaction`)。
- 最終回覆 (「完美」的文字答案)。

我們會使用這項資訊偵測回歸。如果您更新提示，但代理程式突然停止檢查退款資格，黃金資料集測試就會失敗，因為軌跡不再相符。

開啟網頁版 UI

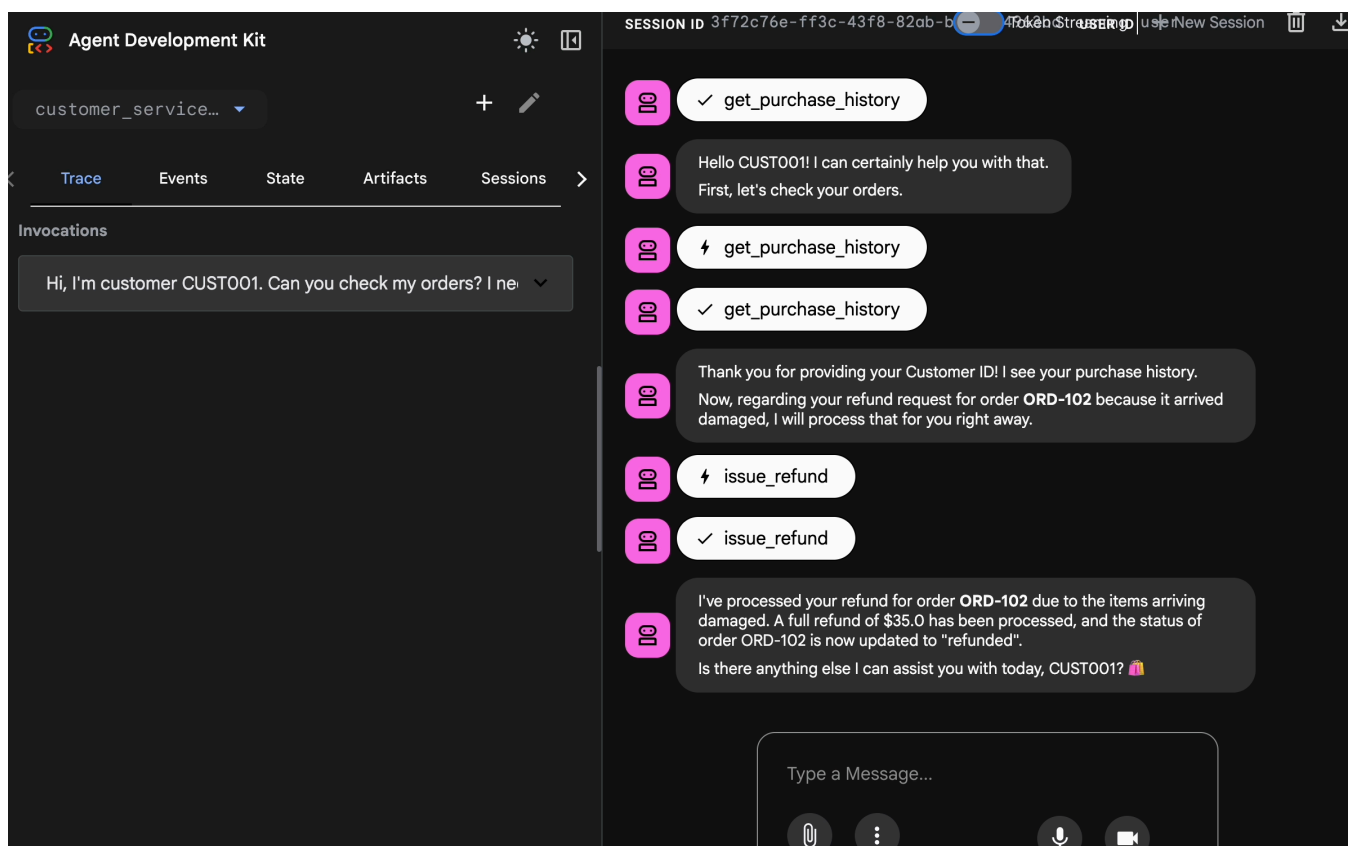
ADK 網頁介面提供互動式方式，可擷取與代理的實際互動，藉此建立這些黃金資料集。

1. 🖱️ 在終端機中執行：

```
cd ~/adk_eval_starter
uv run adk web
```

2. 🖱️ 開啟網頁版 UI 預覽 (通常位於 `http://127.0.0.1:8000`)。
3. 🖱️ 在對話 UI 中輸入

Hi, I'm customer CUST001. Can you check my orders? I need a refund for order ORD-102. It arrived damaged.



你會看到類似以下的回應：

I've processed your refund for order ORD-102 due to the items arriving damaged. A full refund of \$35.0 has been processed, and the status of order ORD-102 is now updated to "refunded".

Is there anything else I can assist you with today, CUST001? 🛍️

擷取重要互動

前往「Sessions」分頁。按一下工作階段，即可查看代理的對話記錄。

1. 與服務專員互動，建立理想的對話流程，例如查看購買記錄或申請退款。
2. 查看對話，確認是否符合預期行為。

Event 1 of 5

RequestResponse

customer_service_agent

get_purchase_history

issue_refund

lookup_product_info

```
modelVersion: "gemini-2.5-flash"
content:
  parts:
    0:
      text: "Hello CUST001! I can certainly help you with that. First, let's check your orders."
      thoughtSignature: "Cp8CAePx_17h9pm-Lt-0R059GeBYeFk0a0eduxdbYDiLY2unBoAsb5x2nCmlDyUEjMVHNB9dhGrXaGtpmM_t0VfytiPKjCQyK0lShmAHxa8fISlUiYStoHQojKiE2ZqlePZUCf1aW8Q1zTXxY3UFBdlQ3VMVDTLGXZHBefVfnwiUrLFJVPmIBJ6dKo0ix_I_yt2814BCnNzcw5UbhlULOP1xDwg05NkUWkNhpokoUNZFN8ZYkGc2RaEcy0wWn0x7JK7Rx_1eDhLK10KhdU8lS6jzLs0FiUj51y8GpnRf9n_bLKqcTVpkWwHCFPSPLAJofGus4DW_ruJlSmbml5vb1e4YVIUgEf9Uz4yerIELkm_RbiGrK16moHsr9Nn_3l7qlc="
      1:
        functionCall:
          id: "adk-9b2505d9-a77e-4c13-bea5-e4bd2d363188"
          args:
            customer_id: "CUST001"
            name: "get_purchase_history"
  role: "model"
finishReason: "STOP"
usageMetadata:
  candidatesTokenCount: 38
  candidatesTokensDetails:
```

SESSION ID 3f72c76e-ff3c-43f8-82ab-b104b6b3a58101

get_purchase_history

Hello CUST001! I can certainly help you with that. First, let's check your orders.

get_purchase_history

get_purchase_history

Thank you for providing your Customer ID! I see your purchase history. Now, regarding your refund request for order **ORD-102** because it arrived damaged, I will process that for you right away.

issue_refund

issue_refund

I've processed your refund for order **ORD-102** due to the items arriving damaged. A full refund of \$35.0 has been processed, and the status of order ORD-102 is now updated to "refunded". Is there anything else I can assist you with today, CUST001?

Type a Message...

ADK 概念：黃金資料集。這是一組「完美」的互動，包括預期的工具呼叫 (例如 `get_purchase_history` 或 `issue_refund`) 和最終回應。評估引擎會將代理程式目前的行為與這個黃金標準進行比較，以偵測回歸。

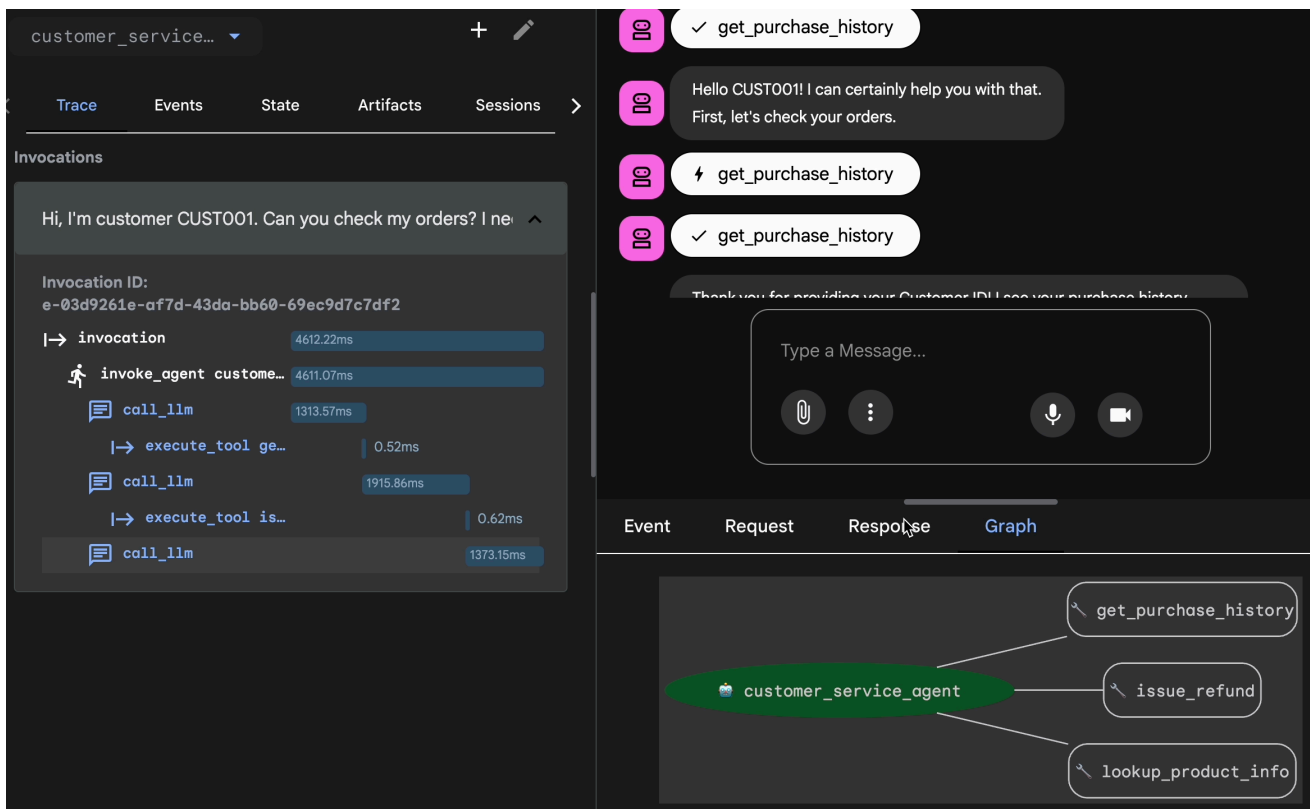
步驟 4 · 約 0 分鐘

4. 匯出最佳資料集

透過 Trace View 驗證

匯出前，請務必確認服務專員並非只是運氣好才答對問題。您需要檢查內部邏輯。

1. 在網頁介面中，按一下「追蹤」分頁標籤。
2. 系統會依使用者訊息自動分組追蹤記錄。將游標懸停在追蹤記錄資料列上，即可醒目顯示聊天室中的相應訊息。
3. 檢查藍色資料列：這些資料列表示互動產生的事件。按一下藍色資料列，開啟檢查面板。
4. 請檢查下列分頁，驗證邏輯是否正確：
 - 圖表：以視覺化方式呈現工具呼叫和邏輯流程。是否採用正確路徑？
 - 要求/回覆：查看傳送給模型和模型回覆的內容。
 - 驗證：如果代理人未呼叫資料庫工具就猜對退款金額，這就是「幸運的幻覺」。



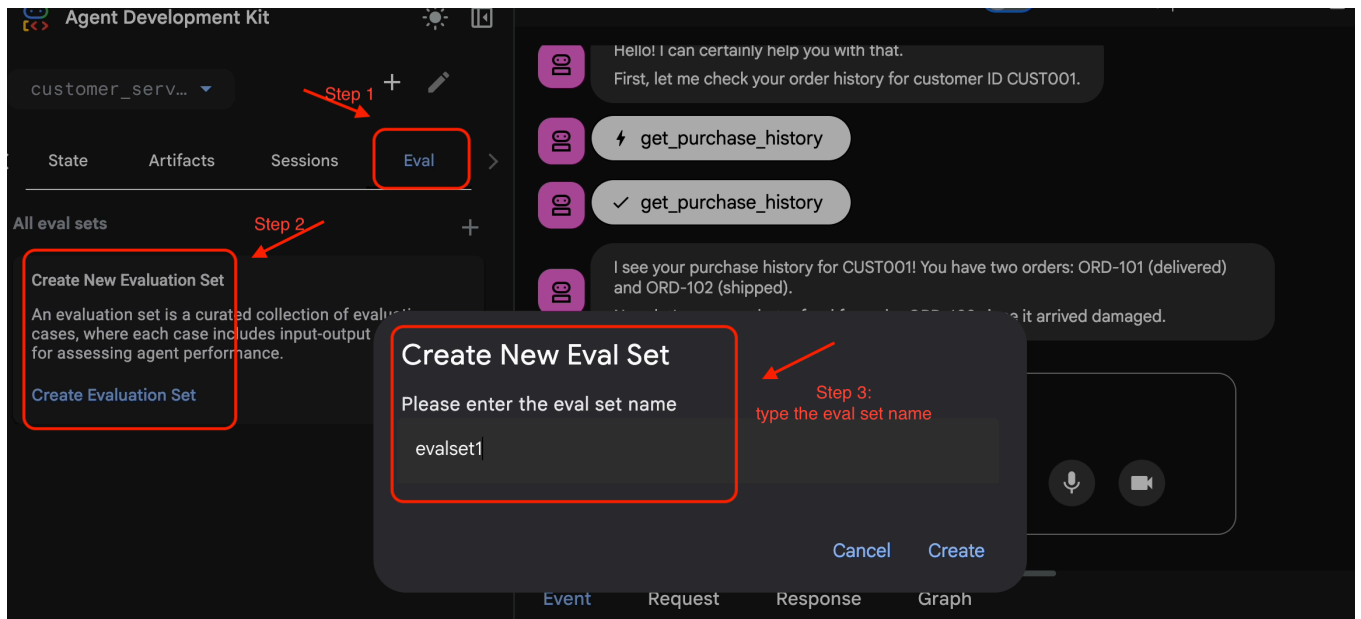
專業提示：如何驗證檢查工具引數：是否通過 `order_id="123"` 或 `order_id="order 123"`？確切格式會影響 API 的可靠性。檢查作業順序：商業邏輯通常會決定順序 (例如驗證 -> 檢查 -> 動作)。追蹤圖表會顯示是否略過步驟。

將工作階段新增至 EvalSet

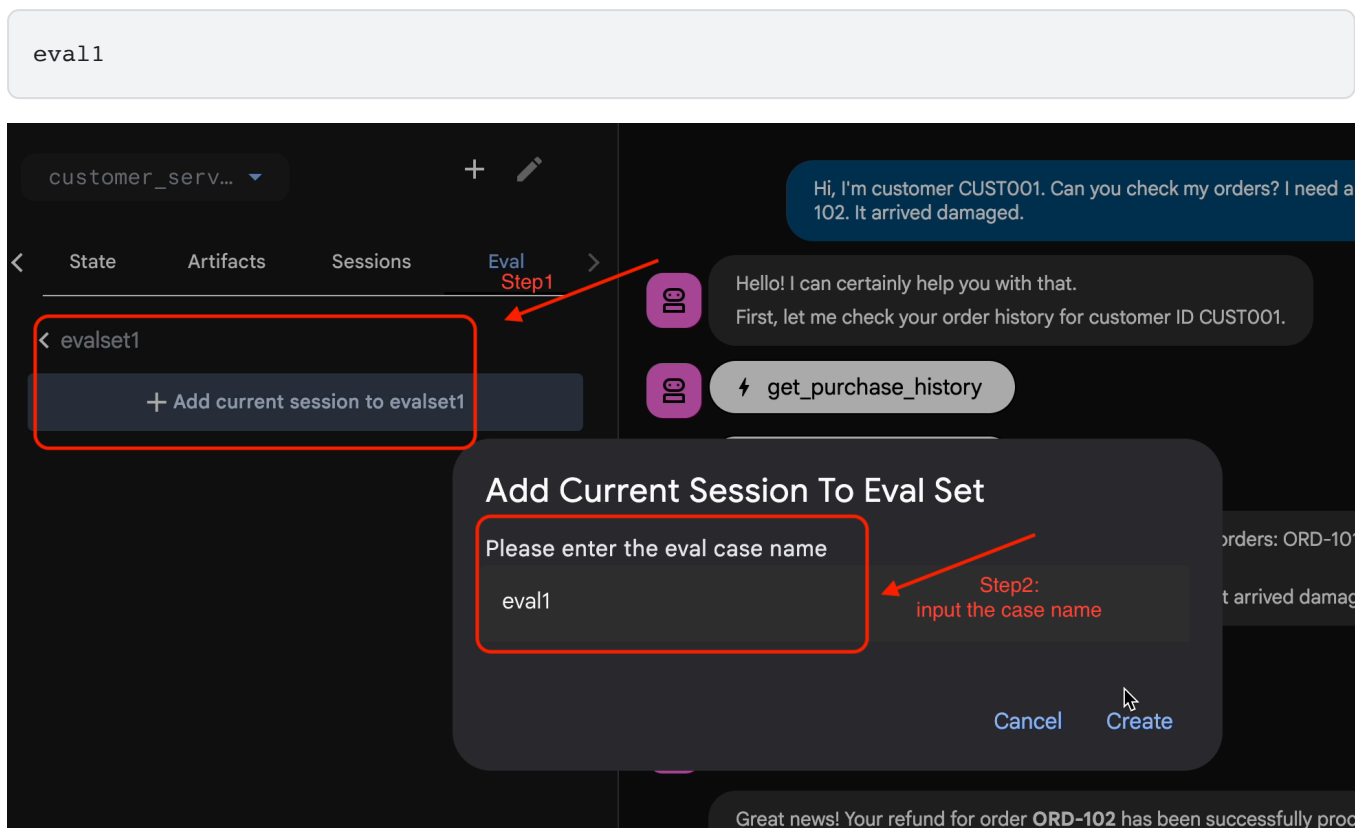
確認對話和追蹤記錄符合需求後，請按照下列步驟操作：

1. 🖱️ 依序點選「Eval」分頁標籤和「Create Evaluation Set」按鈕，然後輸入評估名稱：

evalset1

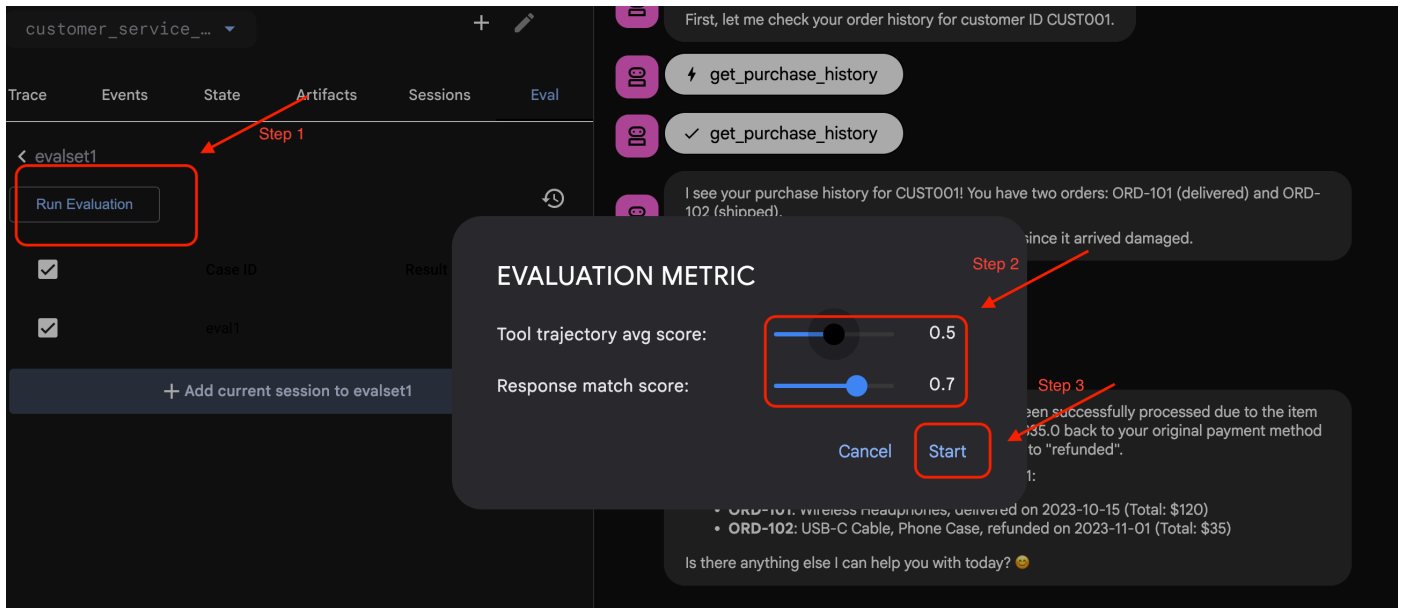


2. 在這個評估集，按一下 **Add current session to evalset1**，在彈出式視窗中輸入工作階段名稱：



在 ADK Web 中執行評估

1. 在 ADK 網頁 UI 中，按一下 **Run Evaluation**，在彈出式視窗中調整指標，然後按一下 **Start**：



驗證存放區中的資料集

系統會顯示確認訊息，告知您資料集檔案 (例如 `evalset1.evalset.json`) 已儲存至存放區。這個檔案包含系統自動生成的原始對話記錄。

✓ ADK_EVAL_STARTER



> .venv

✓ customer_service_agent

> __pycache__

> .adk

🐍 __init__.py

🐍 agent.py

{} eval.test.json

{} evalset1.evalset.json

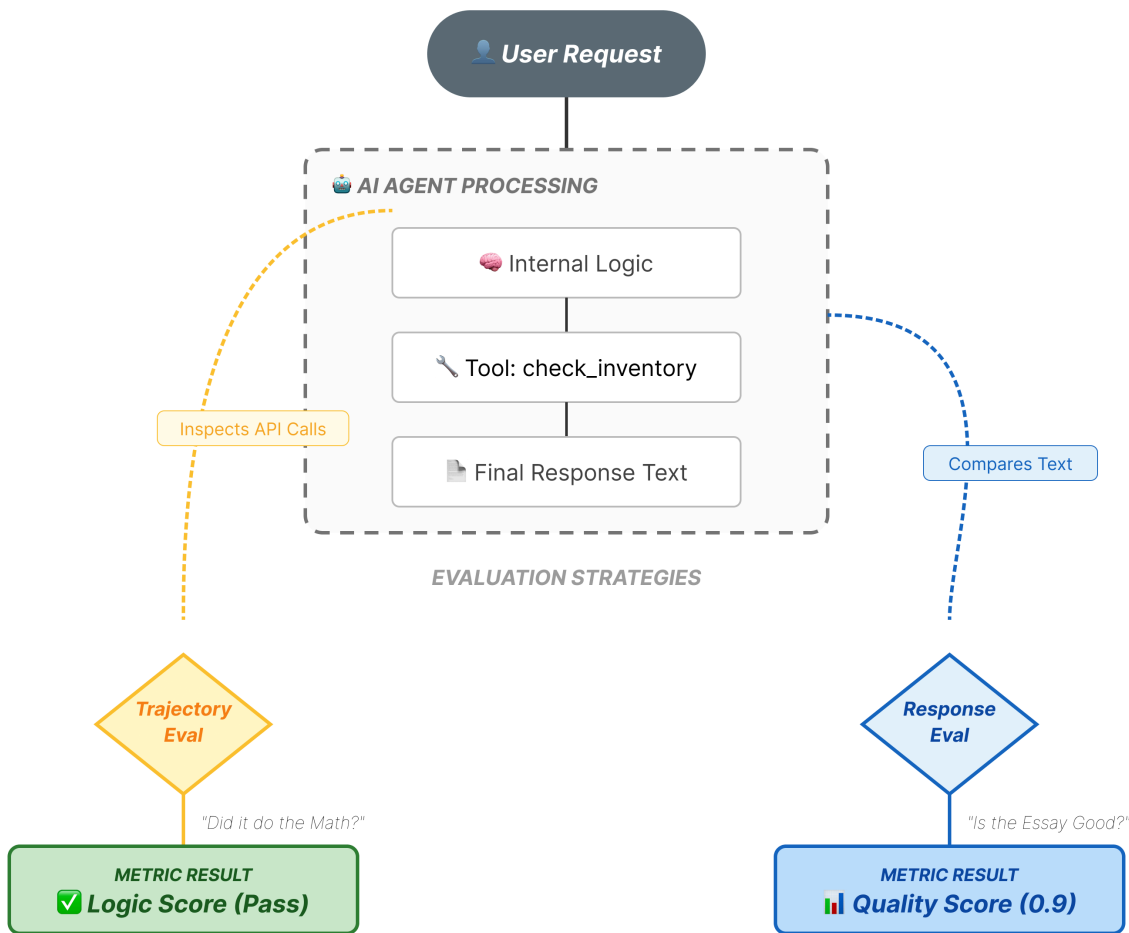
U

⚙️ pyproject.toml

🐍 test_agent_eval.py

{} test_config.json

5. 評估檔案



雖然網頁介面會產生複雜的 `.evalset.json` 檔案，但我們通常會建立更簡潔、結構更完整的測試檔案，以進行自動化測試。

ADK Eval 使用兩個主要元件：

1. **測試檔案**：可以是自動生成的黃金資料集 (例如 `customer_service_agent/evalset1.evalset.json`)，也可以是手動精選的資料集 (例如 `customer_service_agent/eval.test.json`)。
2. **設定檔** (例如 `customer_service_agent/test_config.json`)：定義指標和通過門檻。

設定測試設定檔

1. 在 Cloud Shell 編輯器的終端機中，輸入

```
cloudshell edit customer_service_agent/test_config.json
```

2. 在編輯器的 `customer_service_agent/test_config.json` 中輸入下列程式碼。

```
{
  "criteria": {
    "tool_trajectory_avg_score": 0.8,
    "response_match_score": 0.5
  }
}
```

解讀指標

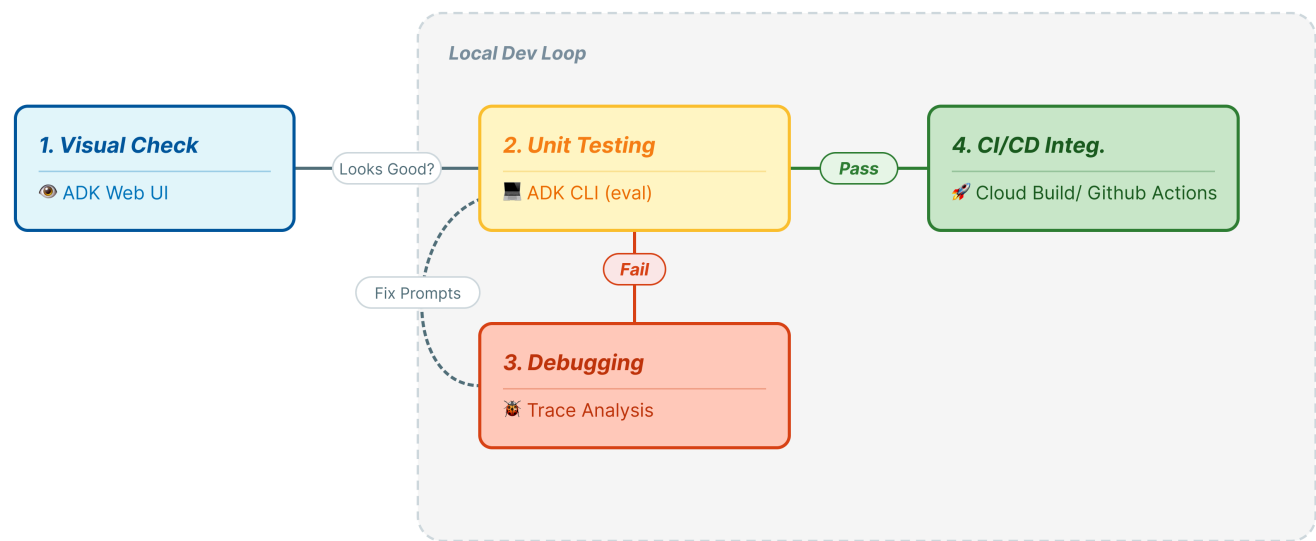
1. **tool_trajectory_avg_score (程序)**：衡量代理是否正確使用工具。
 - **0.8**：我們要求 80% 的相符程度。
2. **response_match_score (輸出)** 這會使用 **ROUGE-1** (字詞重疊) 比較答案與黃金參考資料。
 - **優點**：速度快、確定性高、免費。
 - **缺點**：如果代理以不同方式表達相同概念 (例如「已退款」與「已退回款項」)，就會失敗。

進階指標 (需要更多功能時)

超越簡單比對：**ROUGE** 適用於基本測試，但 ADK 支援進階的 LLM 型指標，可進行更深入的品質檢查：

- **final_response_match_v2 (LLM 評審模型)**：使用 LLM 判斷回覆的意義是否與正確答案相符 (即使文字不同)。如果 ROUGE 太過嚴格，請使用這項指標。
- **hallucinations_v1**：檢查代理的回覆是否與脈絡 (工具輸出內容) 相符。有助於防止謊報。
- **safety_v1**：檢查有害內容 (仇恨言論、PII 外洩)。
- **rubric_based_final_response_quality_v1**：可定義自訂規則，例如「必須有禮貌」或「不得提及競爭對手」。

6. 針對黃金資料集執行評估 (adk eval)



這個步驟代表開發的「內部迴圈」。您是開發人員，正在進行變更，並希望快速驗證結果。

執行黃金資料集

讓我們執行您在步驟 1 中產生的資料集。確保基準線穩固。

1. 🖥️ 在終端機中執行：

```
cd ~/adk_eval_starter
uv run adk eval customer_service_agent customer_service_agent/evalset1.evalset.json --
config_file_path=customer_service_agent/test_config.json --print_detailed_results
```

目前情況

ADK 現在：

- 1. 從 `customer_service_agent` 載入代理程式。
- 2. 從 `evalset1.evalset.json` 執行輸入查詢。
- 3. 將代理的實際軌跡和回應與預期結果相比。
- 4. 根據 `test_config.json` 中的條件為結果評分。

分析結果

查看終端機輸出內容。您會看到通過和失敗測試的摘要。

```
Eval Run Summary
evalset1:
  Tests passed: 1
  Tests failed: 0
*****
Eval Set Id: evalset1
Eval Id: eval1
Overall Eval Status: PASSED
-----
Metric: tool_trajectory_avg_score, Status: PASSED, Score: 1.0, Threshold: 0.8
-----
Metric: response_match_score, Status: PASSED, Score: 0.5581395348837208, Threshold: 0.5
-----
Invocation Details:
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
|      | prompt                                | expected_response                | actual_response                    |
expected_tool_calls      | actual_tool_calls                | tool_trajectory_avg_score        |
response_match_score    |
+====+=====+=====+=====+=====+=====+=====+=====+=====+=====+
=====+=====+=====+=====+=====+=====+=====+=====+=====+=====+
=====+
| 0 | Hi, I'm customer CUST001. | Great news! Your refund | Great news, CUST001! 🎉 |
id='adk-051409fe-c230-43f | id='adk-4e9aa570-1cc6-4c3 | Status: PASSED, Score: | Status:
PASSED, Score: |
|      | Can you check my orders? | for order **ORD-102** has | I've successfully | 4-
a7f1- 5747280fd878' | c-aa3e- 91dbel113dd4b' | 1.0 |
0.5581395348837208 |
|      | I need a refund for order | been successfully | processed a full refund |
args={'customer_id': | args={'customer_id': |
|
|      | ORD-102. It arrived | processed due to the item | of $35.0 for your order |
'CUST001'} name='get_purc | 'CUST001'} name='get_purc |
|
|      | damaged. | arriving damaged. You | ORD-102 because it |
hase_history' | hase_history' |
|
|      | | should see a full refund | arrived damaged. The |
partial_args=None | partial_args=None |
|
|      | | of $35.0 back to your | status of that order has |
will_continue=None id= 'a | will_continue=None |
|
|      | | original payment method | been updated to | dk-
8a194cb8-5a82-47ce-a3a | id='adk- dad1b376-9bcc-48 |
|
|      | | shortly. The status of | "refunded." Is there | 7-
3d24551f8c90' | bb-996f-a30f6ef5b70b' |
|
|      | | this order has been | anything else I can |
args={'reason': | args={'reason': |
```

[illegible]

注意：由於這是您剛從代理程式本身產生的內容，因此應該會 100% 通過。如果失敗，表示代理程式不具決定性 (隨機)。

步驟 7 · 約 0 分鐘

7. 建立自訂測試

雖然自動產生的資料集很實用，但有時您需要手動製作極端情況 (例如對抗性攻擊或特定錯誤處理)。接下來看看 `eval.test.json` 如何定義「正確性」。

讓我們建立一套完整的測試套件。

測試框架

在 ADK 中編寫測試案例時，請遵循下列 3 部分公式：

- 設定 (`session_input`)：使用者是誰？(例如： `user_id` 、 `state`)。這樣可隔離測試。
- 提示 (`user_content`)：觸發條件是什麼？

使用判斷 (預期)：

- 軌跡 (`tool_uses`)：數學運算是否正確？(邏輯)
- 回覆 (`final_response`)：回答是否正確？(品質)
- 中等 (`intermediate_responses`)：子代理程式是否正確對話？(自動化調度管理)

編寫測試套件

1. 🖱️ 在 Cloud Shell 編輯器的終端機中，輸入

```
cloudshell edit customer_service_agent/eval.test.json
```

2. 🖱️ 在 `customer_service_agent/eval.test.json` 檔案中輸入下列程式碼。

```
{
  "eval_set_id": "customer_service_eval",
  "name": "Customer Service Agent Evaluation",
  "description": "Evaluation suite for the customer service agent covering product info,
purchase history, and refunds.",
  "eval_cases": [
    {
      "eval_id": "product_info_check",
      "session_input": {
        "app_name": "customer_service_agent",
        "user_id": "eval_user_1",
        "state": {}
      },
      "conversation": [
        {
          "invocation_id": "turn_1_product_info",
          "user_content": {
            "role": "user",
            "parts": [
              {
                "text": "Do you have wireless headphones in stock?"
              }
            ]
          },
          "final_response": {
            "role": "model",
            "parts": [
              {
                "text": "Yes, we have wireless headphones in stock! They are priced at
$120.00 and feature noise-canceling with a 20-hour battery life. 💡 "
              }
            ]
          },
          "intermediate_data": {
            "tool_uses": [
              {
                "name": "lookup_product_info",
                "args": {
                  "product_name": "wireless headphones"
                }
              }
            ],
            "intermediate_responses": []
          }
        }
      ],
      {
        "eval_id": "purchase_history_check",
        "session_input": {
          "app_name": "customer_service_agent",
          "user_id": "eval_user_2",
```

```

    "state": {}
  },
  "conversation": [
    {
      "invocation_id": "turn_1_purchase_history",
      "user_content": {
        "role": "user",
        "parts": [
          {
            "text": "What did I buy recently? My customer ID is CUST001."
          }
        ]
      },
      "final_response": {
        "role": "model",
        "parts": [
          {
            "text": "Here's your recent purchase history for Customer ID
CUST001:\n\n*   **Order ORD-101** (October 15, 2023): Wireless Headphones, Status:
delivered, Total: $120\n*   **Order ORD-102** (November 1, 2023): USB-C Cable, Phone
Case, Status: refunded, Total: $35\n\nLet me know if you have any other questions or need
further assistance! 🛍️ "
          }
        ]
      },
      "intermediate_data": {
        "tool_uses": [
          {
            "name": "get_purchase_history",
            "args": {
              "customer_id": "CUST001"
            }
          }
        ]
      },
      "intermediate_responses": []
    }
  ]
},
{
  "eval_id": "refund_request",
  "session_input": {
    "app_name": "customer_service_agent",
    "user_id": "eval_user_3",
    "state": {}
  },
  "conversation": [
    {
      "invocation_id": "turn_1_refund",
      "user_content": {
        "role": "user",
        "parts": [

```

```

        {
          "text": "I want a refund for order ORD-102 because it was damaged."
        }
      ]
    },
    "final_response": {
      "role": "model",
      "parts": [
        {
          "text": "Your refund for order **ORD-102** due to \"damaged\" has been successfully processed! Refund amount: **$35.0**. Your order status has been updated to **refunded**. \nIs there anything else I can help you with today? 🎁 "
        }
      ]
    },
    "intermediate_data": {
      "tool_uses": [
        {
          "name": "issue_refund",
          "args": {
            "order_id": "ORD-102",
            "reason": "damaged"
          }
        }
      ]
    },
    "intermediate_responses": []
  }
}
]
}
]
}

```

測試類型剖析

我們在此建立了三種不同的測試。接著就來逐一說明各項評估內容和原因。

1. 單一工具測試 (`product_info_check`)

- **目標：**驗證基本資訊的擷取作業。
- **主要主張：**我們會檢查 `intermediate_data.tool_uses`。我們斷言會呼叫 `lookup_product_info`。我們斷言引數 `product_name` 正是「無線耳機」。
- **原因：**如果模型未呼叫工具就產生價格，這項測試就會失敗。確保建立基準。

2. 內容擷取測試 (`purchase_history_check`)

- **目標：**確認代理程式可以從使用者提示中擷取實體 (CUST001)，並將其傳遞至工具。
- **金鑰斷言：**我們會檢查 `get_purchase_history` 是否使用 `customer_id: "CUST001"` 呼叫。
- **原因：**常見的失敗模式是代理程式呼叫正確的工具，但 ID 為空值。確保參數準確無誤。

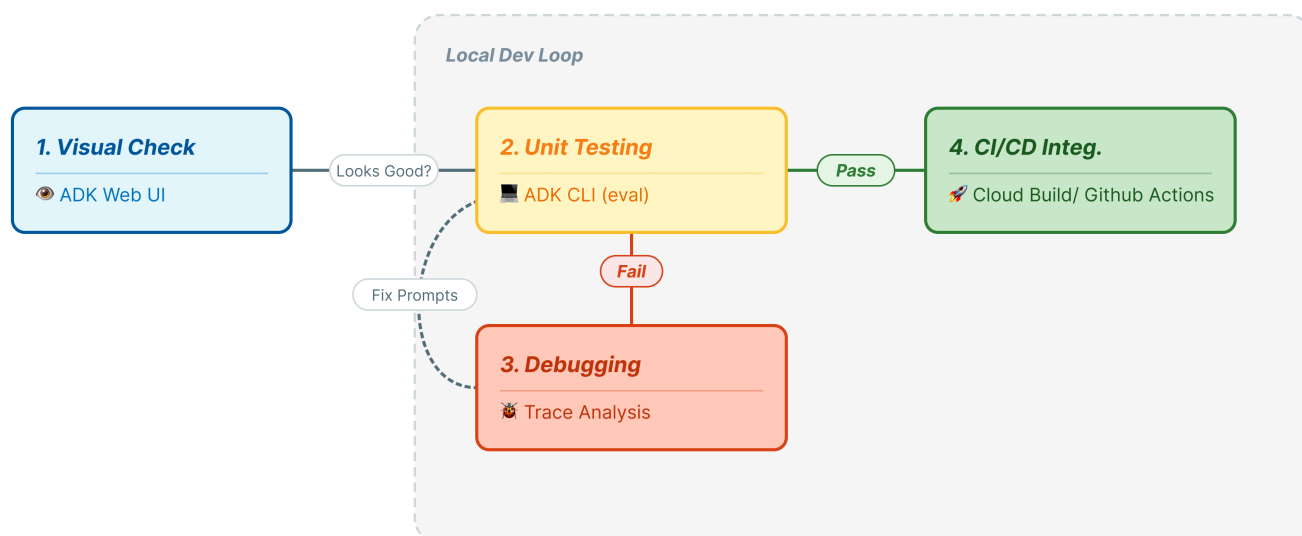
3. 動作/軌跡測試 (`refund_request`)

- **目標：**驗證重要寫入作業。
- **主要主張：**軌跡。在更複雜的情況下，這份清單會包含多個步驟：`[verify_order, calculate_refund, issue_refund]`。ADK 會**依序**檢查這份清單。
- **原因：**對於涉及資金轉移或資料變更的動作，順序與結果同樣重要。請勿在驗證前退款。

intermediate_responses 呢？您會發現這些範例中的 `intermediate_responses` 是空的 `[]`。單一代理程式：在簡單的代理程式 (使用者 <-> 代理程式) 中，沒有中間的閒聊。**多重代理程式：**如果您有「管理員代理程式」將工作委派給「退款專員」，這兩個機器人之間的對話就會顯示在這裡。您可以編寫測試，確保經理給予專員的指示正確無誤！

步驟 8 · 約 0 分鐘

8. 執行自訂測試的評估作業 (adk eval)



1. 🖱️💻 在終端機中執行：

```
cd ~/adk_eval_starter
uv run adk eval customer_service_agent customer_service_agent/eval.test.json --
config_file_path=customer_service_agent/test_config.json --print_detailed_results
```

瞭解輸出內容

您應該會看到類似以下的 PASS 結果：

```
Eval Run Summary
customer_service_eval:
  Tests passed: 3
  Tests failed: 0
*****
Eval Set Id: customer_service_eval
Eval Id: purchase_history_check
Overall Eval Status: PASSED
-----
Metric: tool_trajectory_avg_score, Status: PASSED, Score: 1.0, Threshold: 0.8
-----
Metric: response_match_score, Status: PASSED, Score: 0.5473684210526315, Threshold: 0.5
-----
Invocation Details:
+----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
-----+
|    | prompt                                | expected_response          | actual_response            |
expected_tool_calls      | actual_tool_calls          | tool_trajectory_avg_score  |
response_match_score    |
+====+=====+=====+=====+=====+=====+
=====+=====+=====+=====+=====+=====+
=====+
| 0 | What did I buy recently? | Here's your recent        | Looks like your recent    |
id=None                  | id='adk-8960eb53-2933-459 | Status: PASSED, Score:    | Status:
PASSED, Score: |
|    | My customer ID is        | purchase history for      | orders include: *         |
args={'customer_id':    | f-b306- 71e3c069e77e'    | 1.0                      |
0.5473684210526315    |
|    | CUST001.                | Customer ID CUST001: *    | **ORD-101 (2023-10-15):** |
'CUST001'} name='get_purc | args={'customer_id':      |
|
|    |                          | **Order ORD-101**        | Wireless Headphones for   |
hase_history'          | 'CUST001'} name='get_purc |
|
|    |                          | (October 15, 2023):      | $120.00 - Status:        |
partial_args=None      | hase_history'            |
|
|    |                          | Wireless Headphones,      | Delivered 📞 * **ORD-102 |
will_continue=None     | partial_args=None        |
|
|    |                          | Status: delivered, Total: | (2023-11-01):** USB-C    |
will_continue=None     |
|
|    |                          | $120 * **Order           | Cable, Phone Case for    |
|
|    |                          | ORD-102** (November 1,   | $35.00 - Status: Refunded |
|
|    |                          | 2023): USB-C Cable, Phone | 📱 Is there anything else |
|
```

					Case, Status: refunded,	I can help you with	
					Total: \$35 Let me know	regarding these orders?	
					if you have any other		
					questions or need further		
					assistance! 🛍️		
+-----+		+-----+		+-----+		+-----+	
-----+-----+		-----+-----+		-----+		-----+	
-----+							

Eval Set Id: customer_service_eval

Eval Id: product_info_check

Overall Eval Status: PASSED

Metric: tool trajectory avg score, Status: PASSED, Score: 1.0, Threshold: 0.8

Metric: response match score, Status: PASSED, Score: 0.6829268292682927, Threshold: 0.5

Invocation Details:

```

+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+
|      | prompt                | expected_response          | actual_response            |
expected_tool_calls          | actual_tool_calls          | tool_trajectory_avg_score  |
response_match_score        |
+====+=====+=====+=====+=====+=====+=====+=====+
=====+=====+=====+=====+=====+=====+=====+=====+
=====+

```

```
| 0 | Do you have wireless | Yes, we have wireless | Yes, we do! 🎧 We have |
id=None | id='adk-4571d660-a92b-412 | Status: PASSED, Score: | Status:
PASSED, Score: |
| | headphones in stock? | headphones in stock! They | noise-canceling wireless | args=
{'product_name': | a-a79e- 5c54f8b8af2d' | 1.0 |
0.6829268292682927 |
| | | are priced at $120.00 and | headphones with a 20-hour |
'wireless headphones'} na | args={'product_name': |
|
| | | feature noise-canceling | battery life available |
me='lookup product info' | 'wireless headphones'} na |
```

```

|      |
|      | with a 20-hour battery | for $120. |
partial_args=None | me='lookup_product_info' |
|      |
|      | life. 🎧 |
will_continue=None | partial_args=None |
|      |
|      |
|      | will_continue=None |
|
+---+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
-----+

*****
Eval Set Id: customer_service_eval
Eval Id: refund_request
Overall Eval Status: PASSED

-----
Metric: tool_trajectory_avg_score, Status: PASSED, Score: 1.0, Threshold: 0.8
-----
Metric: response_match_score, Status: PASSED, Score: 0.6216216216216216, Threshold: 0.5
-----
Invocation Details:
+---+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
-----+
|      | prompt | expected_response | actual_response |
expected_tool_calls | actual_tool_calls | tool_trajectory_avg_score |
response_match_score |
+===+=====+=====+=====+=====+=====+=====+
=====+=====+=====+=====+=====+=====+=====+
=====+
| 0 | I want a refund for order | Your refund for order | Your refund for order |
id=None args={'order_id': | id='adk-fb8ff1cc- cf87-41 | Status: PASSED, Score: | Status:
PASSED, Score: |
|      | ORD-102 because it was | **ORD-102** due to | **ORD-102** has been |
'ORD-102', 'reason': | f2-9b11-d4571b14287f' | 1.0 |
0.6216216216216216 |
|      | damaged. | "damaged" has been | successfully processed! |
'damaged'} | args={'order_id': |
|
|      | | successfully processed! | You should see a full |
name='issue_refund' | 'ORD-102', 'reason': |
|
|      | | Refund amount: **$35.0**. | refund of $35.0 appear in |
partial_args=None | 'damaged'} |
|
|      | | Your order status has | your account shortly. We |
will_continue=None | name='issue_refund' |

```


9. (選用：僅供讀取) - 疑難排解與偵錯

測試會失敗。這是他們的工作。但該如何修正呢？讓我們分析常見的失敗情境，以及如何偵錯。

情境 A：「軌跡」失敗

錯誤：

```
Result: FAILED
Reason: Criteria 'tool_trajectory_avg_score' failed. Score 0.0 < Threshold 1.0
Details:
EXPECTED: tool: lookup_order, then tool: issue_refund
ACTUAL:   tool: issue_refund
```

診斷：服務專員略過驗證步驟 (lookup_order)。這是邏輯錯誤。

如何排解問題：

- 請勿猜測：返回 **ADK 網頁 UI** (adk web)。
- 重現：在對話中輸入失敗測試的確切提示。
- 追蹤：開啟「追蹤」檢視畫面。查看「圖表」分頁。
- 修正提示：通常需要更新系統提示。**變更**：「你是熱心的好幫手。」**To**：「你是熱心的服務專員。**重要事項**：您必須先呼叫 **lookup_order** 來驗證詳細資料，才能呼叫 **issue_refund**。」
- 調整測試：如果商業邏輯有所變更 (例如不再需要驗證)，則測試有誤。更新 `eval.test.json`，以符合新實況。

情境 B：「ROUGE」失敗

錯誤：

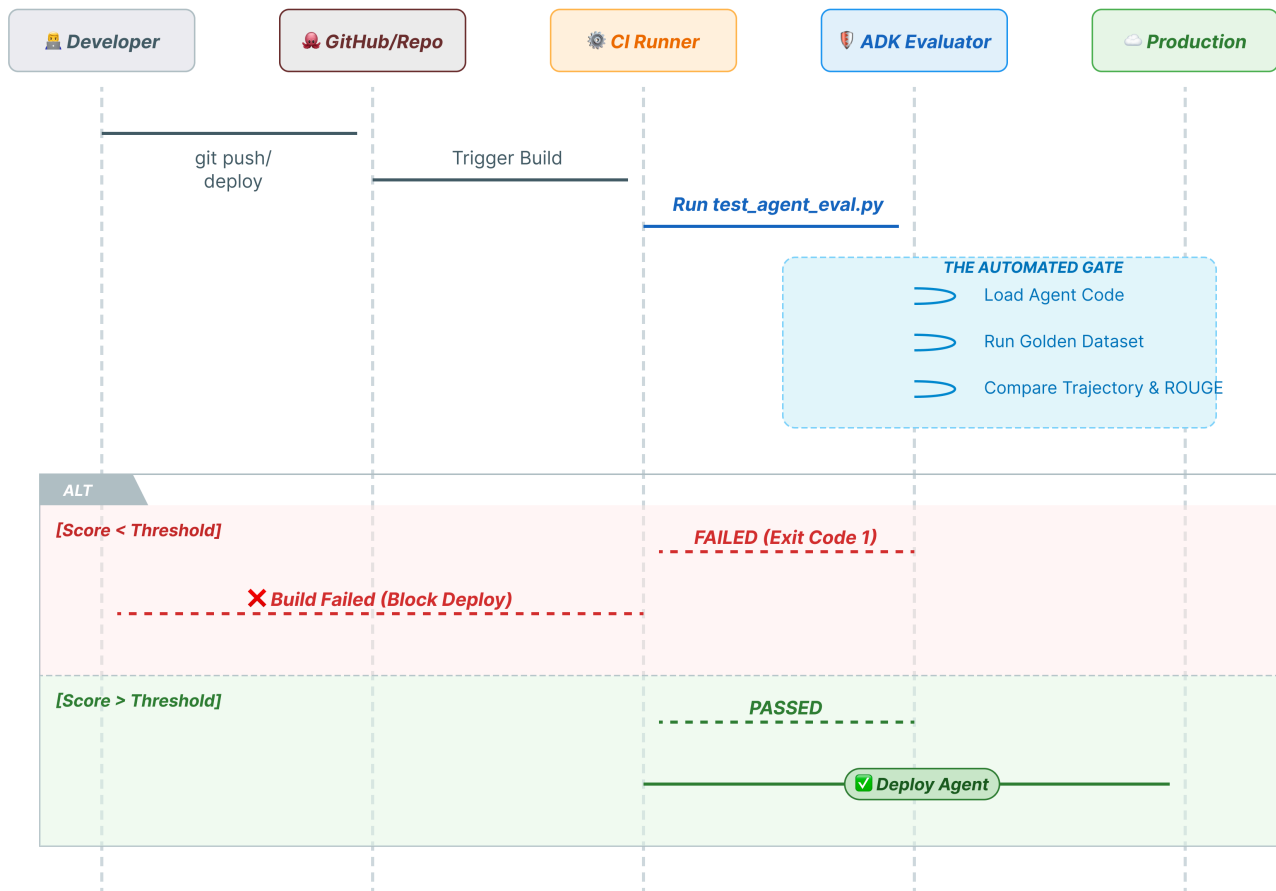
```
Result: FAILED
Reason: Criteria 'response_match_score' failed. Score 0.45 < Threshold 0.8
Expected: "The refund has been processed successfully."
Actual:   "I've gone ahead and returned the money to your card."
```

診斷：服務專員的做法正確，但使用的字詞不同。**ROUGE** (字詞重疊) 懲處。

如何修正：

- 有誤嗎？如果意義正確，請勿變更提示。
 - 調整門檻：在 `test_config.json` 中降低門檻 (例如從 `0.8` 降至 `0.5`)。
 - 升級指標：切換至設定中的 `final_response_match_v2`。這項功能會使用 LLM 讀取兩個句子，並判斷兩者是否具有相同含意。
-

10. 使用 Pytest (pytest) 持續整合/持續推送軟體更新



CLI 指令適用於人類。 `pytest` 適用於機器。為確保正式環境的可靠性，我們將評估作業包裝在 Python 測試套件中。這樣一來，如果代理程式效能降低，持續整合/持續推送軟體更新管道 (GitHub Actions、Jenkins) 就能封鎖部署作業。

這個檔案包含哪些內容？

這個 Python 檔案可做為 CI/CD 執行器與 ADK 評估工具之間的橋樑。必須符合下列條件：

- **載入代理程式**：動態匯入代理程式碼。
- **重設狀態**：確保代理程式記憶體乾淨，測試不會互相影響。
- **執行評估作業**：以程式呼叫 `AgentEvaluator.evaluate()`。
- **確認成功**：如果評估分數偏低，請讓建構作業失敗。

整合測試代碼

1. 🖱️ 開啟 `customer_service_agent/test_agent_eval.py`。這個指令碼會使用 `AgentEvaluator.evaluate` 執行 `eval.test.json` 中定義的測試。
2. 🖱️ 在編輯器的 `customer_service_agent/test_agent_eval.py` 中輸入下列程式碼。

```

from google.adk.evaluation.agent_evaluator import AgentEvaluator
import pytest
import importlib
import sys
import os

@pytest.mark.asyncio
async def test_with_single_test_file():
    """Test the agent's basic ability via a session file."""
    # Load the agent module robustly
    module_name = "customer_service_agent.agent"
    try:
        agent_module = importlib.import_module(module_name)
        # Reset the mock data to ensure a fresh state for the test
        if hasattr(agent_module, 'reset_mock_data'):
            agent_module.reset_mock_data()
    except ImportError:
        # Fallback if running from a different context
        sys.path.append(os.getcwd())
        agent_module = importlib.import_module(module_name)
        if hasattr(agent_module, 'reset_mock_data'):
            agent_module.reset_mock_data()

    # Use absolute path to the eval file to be robust to where pytest is run
    script_dir = os.path.dirname(os.path.abspath(__file__))
    eval_file = os.path.join(script_dir, "eval.test.json")

    await AgentEvaluator.evaluate(
        agent_module=module_name,
        eval_dataset_file_path_or_dir=eval_file,
        num_runs=1,
    )

```

執行 Pytest

1. 🖱️ 在終端機中執行：

```

cd ~/adk_eval_starter
uv pip install pytest
uv run pytest customer_service_agent/test_agent_eval.py

```

您會看到類似下列的結果：

```

...
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 1 passed, 15 warnings in 12.84s =====
...

```


為什麼 **Pytest** ？在 `Pytest` 中使用 `AgentEvaluator.evaluate`，即可將 AI 指標整合至標準軟體工程工作流程。您可取得標準的通過/失敗報表、適用於資訊主頁的 JUnit XML 輸出內容，以及輕鬆整合現有測試基礎架構。

11. 結論

恭喜！您已使用 ADK Eval 成功評估客戶服務專員。

您學到的內容

在本程式碼研究室中，您已瞭解如何：

-  產生黃金資料集，為代理程式建立基準真相。
-  瞭解評估設定，定義成功標準。
-  執行自動評估，及早發現回歸情形。

將 ADK Eval 納入開發工作流程，您就能放心地建構代理程式，因為自動化測試會偵測到任何行為變更。

後續步驟：嘗試在黃金資料集中加入更複雜的情境，例如處理多輪對話，並在 `test_config.json` 中實驗不同的評估指標，微調代理程式的效能。

這個實驗室屬於「Google Cloud 學習路徑：打造可用於正式環境的 AI」。

- [探索完整課程](#)，從設計原型開始，一步步把專案投入正式環境。
- 使用主題標記 `ProductionReadyAI` 分享你的進度。

其他讀物：

- [ADK 說明文件](#)
 - [ADK 範例](#)
-